

```
def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1
    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()
    for obj in self.objects:          # BUG: mutating this list
        while iterating over it
            if is_caught(basket_rect, obj):
                self.score += 1
                self.objects.remove(obj)

    missed = [o for o in self.objects if
o.is_past_bottom(HEIGHT)]
    if missed:
        self.objects = [o for o in self.objects if not
o.is_past_bottom(HEIGHT)]
        self.misses += len(missed)
        if self.misses >= MAX_MISSES:
            self.game_over = True

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.basket, self.objects)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10,
10))
    renderer.draw_text(surface, font, f"Misses:
{self.misses}/{MAX_MISSES}", (10, 36))

    if self.game_over:
        renderer.draw_banner(surface, font, f"Game Over! Final
score: {self.score}. Press R to restart.")
```

```
"""
```

```
collision: figures out whether a falling object is within the basket.
```

```
"""
```

```
def is_caught(basket_rect, obj):
```

```
    return basket_rect.left <= obj.x <= basket_rect.right
```

I'm working on a Pygame catch game. Above are

`game/collision.py` and the `update()` method from

`game/game_engine.py`. Two bugs: (1) objects get caught the

moment they spawn at the top, as soon as the basket is under

their x position. (2) When two objects are close together in the

list, one sometimes gets skipped. Explain what causes each bug,

then give me the corrected code for both.

You have **two separate bugs**, and both come from the collision/removal logic.

1. Objects are caught immediately after spawning

Your current collision check is:

```
</> Python
```

```
return basket_rect.left <= obj.x <= basket_rect.right
```

This checks **only the X coordinate**. So even when the object is still at the very top of the screen, it counts as caught as soon as its `x` falls over the basket.

You need to check the object's **vertical position too**—i.e. whether it has reached the basket's height.

2. One object gets skipped

This is caused by:

```
</> Python
```

```
for obj in self.objects:
    if is_caught(basket_rect, obj):
        self.objects.remove(obj)
```

You're **removing items from the same list you're currently iterating over**.

For example:

</> Plain text

```
objects = [A, B, C]
```

```
iteration → A
```

```
A is caught → remove A
```

```
list becomes → [B, C]
```

```
Python moves to the next index → C
```

So B gets skipped.

The clean fix is to iterate over a copy:

</> Python

```
for obj in self.objects[:]:
```

or, even better, rebuild the list using a filtering approach.

Corrected collision.py

Assuming your falling objects have `x` , `y` , and a size/height that can be represented by a rectangle:

</> Python

```
"""
```

```
collision: figures out whether a falling object is within the basket.
```

```
"""
```

```
def is_caught(basket_rect, obj):
```

```
    obj_rect = obj.get_rect()
```

```
    # Object must horizontally overlap the basket
```

```
    horizontal_overlap = (
```

```
        obj_rect.right >= basket_rect.left
```

```
        and obj_rect.left <= basket_rect.right
```

```
)
```

```
    # Object must have reached the basket vertically
```

```
    vertical_overlap = (
```

```
        obj_rect.bottom >= basket_rect.top
```

```
        and obj_rect.top <= basket_rect.bottom
```

```
)
```

This is much safer than checking only `obj.x` .

Corrected `game_engine.py`

`</>` Python

```
def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    # Update all falling objects
    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()

    # Iterate over a COPY of the list so removing objects
    # does not cause other objects to be skipped.
    for obj in self.objects[:]:
        if is_caught(basket_rect, obj):
```

The key changes

Bug 1:

Before:

`</>` Python

```
return basket_rect.left <= obj.x <= basket_rect.right
```

After:

`</>` Python

```
obj_rect = obj.get_rect()

horizontal_overlap = (
    obj_rect.right >= basket_rect.left
    and obj_rect.left <= basket_rect.right
)
```

```

vertical_overlap = (
    obj_rect.bottom >= basket_rect.top
    and obj_rect.top <= basket_rect.bottom
)

return horizontal_overlap and vertical_overlap

```

Now an object at the top **cannot be caught just because its X coordinate matches the basket**.

Bug 2:

Before:

</> Python

```
for obj in self.objects:
```

while doing:

</> Python

```
self.objects.remove(obj)
```

After:

</> Python

```
for obj in self.objects[:]:
```

`self.objects[:]` creates a **shallow copy**, so you can safely remove objects from the original list while iterating.

One more thing: this assumes your object class has a `get_rect()` method. If it doesn't, send me the falling-object class (e.g. `object.py`), and I can give you the exact collision code for your implementation.

this is game.py """

GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic

level (Tasks 2 and 3 ask you to improve them), there's no speed boost

yet (Task 4 builds it from scratch), and catch detection has two known bugs (see game/collision.py and the catch-checking loop below)

that Task 1 asks you to fix.

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
```

```
MAX_MISSES = 5
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
```

```
        self.objects = []
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
        self.misses = 0
```

```
        self.game_over = False
```

```
    def _spawn_object(self):
```

```
        x = random.randint(20, WIDTH - 20)
```

```
        self.objects.append(FallingObject(x=x, y=-14, speed=3))
```

```
    def handle_input(self, keys_pressed):
```

```
        if self.game_over:
```

```
            return
```

```
        if keys_pressed[pygame.K_LEFT]:
```

```
            self.basket.x -= self.basket.speed
```

```
        if keys_pressed[pygame.K_RIGHT]:
```

```
            self.basket.x += self.basket.speed
```

```
        # Boundary handling: only clamps against the screen edges,
```

```
not
```

```
# accounting for the basket's own width - it can hang half
off

# either side of the screen.
self.basket.x = max(0, min(WIDTH, self.basket.x))

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
        return

    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    # Update all falling objects
    for obj in self.objects:
        obj.update()

    basket_rect = self.basket.get_rect()

    # Iterate over a COPY of the list so removing objects
    # does not cause other objects to be skipped.
    for obj in self.objects[:]:
        if is_caught(basket_rect, obj):
            self.score += 1
            self.objects.remove(obj)

    # Find objects that have gone past the bottom
    missed = [
        obj for obj in self.objects
        if obj.is_past_bottom(HEIGHT)
    ]

    if missed:
        self.objects = [
```

```
        obj for obj in self.objects
        if not obj.is_past_bottom(HEIGHT)
    ]

    self.misses += len(missed)

    if self.misses >= MAX_MISSES:
        self.game_over = True

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.basket,
        self.objects
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Misses: {self.misses}/{MAX_MISSES}",
        (10, 36)
    )

    if self.game_over:
        renderer.draw_banner(
            surface,
            font,
            f"Game Over! Final score: {self.score}. Press R to restart."
        )
```

Yes — I can see the issue. Your **logic is mostly correct now**, but the code you pasted has an **indentation problem**: `update()` and `draw()` need to remain inside the `GameEngine` class.

Also, because your `FallingObject` implementation isn't shown, I want to avoid assuming it has `get_rect()`. Based on your existing code, we can fix the two requested bugs cleanly using the object's `x / y` coordinates.

Correct `game.py`

Replace your file with this:

`</>` Python

"""

`GameEngine`: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic level (Tasks 2 and 3 ask you to improve them), there's no speed boost yet (Task 4 builds it from scratch), and catch detection has two known bugs (see `game/collision.py` and the catch-checking loop below) that Task 1 asks you to fix.

"""

`import random`
`import pygame`

`from game.basket import Basket`
`from game.falling_object import FallingObject`
`from game.collision import is_caught`
`from game.renderer import WIDTH, HEIGHT`

`SPAWN_INTERVAL_FRAMES = 50`
`MAX_MISSES = 5`

But the important fix is actually in `collision.py`

Your original:

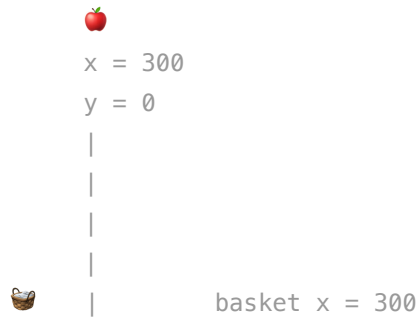
`</>` Python

```
def is_caught(basket_rect, obj):
    return basket_rect.left <= obj.x <= basket_rect.right
```

only checks **X**.

So imagine:

`</>` Plain text



The apple is nowhere near the basket vertically, but because:

</> Plain text

```
basket.left <= apple.x <= basket.right
```

is true, the game says:

"Caught!"

We need to check **both X and Y**.

However, I need to see your `falling_object.py` and `basket.py` to give you the **exact** collision implementation, because their dimensions determine where the collision should occur.

Send me these two files:

</> Plain text

```
game/falling_object.py
game/basket.py
```

Then I'll give you the exact final `collision.py` without guessing about the object dimensions.

Here is `handle_input` from my Pygame game and my `Basket` class. The basket's x is its center. Right now it can move half off either side of the screen. Fix the boundary handling so the whole basket always stays on screen, and make sure movement is smooth when a key is held down. Explain your changes.

"""

Basket: the player-controlled catcher at the bottom of the screen.

"""

```
import pygame
```

```
class Basket:
```

```
    def __init__(self, x, y, width=90, height=24, speed=5):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.speed = speed
```

```
        self.boosted_frames = 0
```

```
        self.normal_speed = speed
```

```
        self.boost_speed = speed
```

```
    def get_rect(self):
```

```
        return pygame.Rect(
```

```
            int(self.x - self.width / 2), int(self.y - self.height / 2),
```

```
            self.width, self.height,
```

```
        )\
```

Yes. Since `self.x` is the **center** of the basket, the boundary calculation needs to account for half the basket width.

Corrected `handle_input`

</> Python

```
def handle_input(self, keys_pressed):
```

```
    if self.game_over:
```

```
        return
```

```
    # Move continuously while the key is held down
```

```
    if keys_pressed[pygame.K_LEFT]:
```

```
        self.basket.x -= self.basket.speed
```

```
    if keys_pressed[pygame.K_RIGHT]:
```

```
        self.basket.x += self.basket.speed
```

```
    # Keep the entire basket inside the screen.
```

```
    half_width = self.basket.width / 2
```

```
    self.basket.x = max(
```

```
        half_width,
```

```
        min(WIDTH - half_width, self.basket.x)
    )
```

Why this fixes it

Your basket is defined like this:

`</>` Python

```
pygame.Rect(
    int(self.x - self.width / 2),
    int(self.y - self.height / 2),
    self.width,
    self.height,
)
```

So if:

`</>` Plain text

```
WIDTH = 800
basket.width = 90
```

the basket's center cannot go all the way to `0` or `800` .

Its center must stay between:

`</>` Plain text

```
45 and 755
```

because:

`</>` Plain text

```
half_width = 90 / 2 = 45

left boundary  = 45
right boundary = 800 - 45 = 755
```

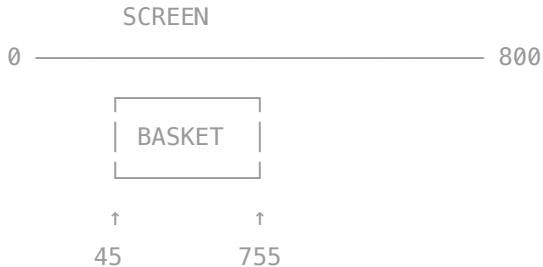
Therefore:

`</>` Python

```
self.basket.x = max(half_width,
                    min(WIDTH - half_width, self.basket.x))
```

guarantees:

</> Plain text



The **entire basket** remains visible.

Smooth movement

You're already using the right approach:

</> Python

```
if keys_pressed[pygame.K_LEFT]:
    self.basket.x -= self.basket.speed

if keys_pressed[pygame.K_RIGHT]:
    self.basket.x += self.basket.speed
```

`pygame.key.get_pressed()` gives you the current state of the keyboard, so this code runs every game frame while the key is held.

That's different from `handle_keydown()`, which only responds when a key-down event occurs.

So the intended structure is:

</> Plain text

```
Every frame
↓
handle_input()
↓
Is LEFT held? → move left
Is RIGHT held? → move right
↓
Clamp basket position
↓
```

```
update()
```

```
↓
```

```
draw()
```

One small improvement: if you want the basket to move at the same physical speed regardless of FPS, the next step would be to use `dt` (delta time) rather than a fixed `speed=5` pixels per frame. For your current game/task, though, the above is the appropriate fix.

[give full code](#)

Here is the **full corrected `game.py`** with smooth key-held movement and the basket kept completely inside the screen.

Corrected game.py



```
"""
```

```
GameEngine: owns the basket and all falling objects.
```

```
Starter version: basket movement and spawning both work at a basic
level (Tasks 2 and 3 ask you to improve them), there's no speed boost
yet (Task 4 builds it from scratch), and catch detection has two
known bugs (see game/collision.py and the catch-checking loop below)
that Task 1 asks you to fix.
```

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
```

```
MAX_MISSES = 5
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.basket = Basket(
```

```
            x=WIDTH / 2,
```

```
        y=HEIGHT - 30
    )

    self.objects = []
    self.frames_until_spawn = 0
    self.score = 0
    self.misses = 0
    self.game_over = False

def _spawn_object(self):
    x = random.randint(20, WIDTH - 20)

    self.objects.append(
        FallingObject(
            x=x,
            y=-14,
            speed=3
        )
    )

def handle_input(self, keys_pressed):
    if self.game_over:
        return

    # Move continuously while the key is held down.
    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed

    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed

    # Keep the entire basket inside the screen.
    #
    # basket.x is the CENTER of the basket, so the center
    # cannot be closer to the edge than half the basket width.
    half_width = self.basket.width / 2

    self.basket.x = max(
        half_width,
        min(WIDTH - half_width, self.basket.x)
```

```
)

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
        return

    # Spawn objects
    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    # Update all falling objects
    for obj in self.objects:
        obj.update()

    # Check for caught objects
    basket_rect = self.basket.get_rect()

    # Iterate over a COPY of the list.
    # This prevents objects from being skipped when one
    # is removed from the original list.
    for obj in self.objects[:]:
        if is_caught(basket_rect, obj):
            self.score += 1
            self.objects.remove(obj)

    # Find objects that have passed the bottom
    missed = [
        obj
        for obj in self.objects
        if obj.is_past_bottom(HEIGHT)
    ]

    if missed:
```

```
# Remove all missed objects
self.objects = [
    obj
    for obj in self.objects
    if not obj.is_past_bottom(HEIGHT)
]

# Increase miss count
self.misses += len(missed)

# End game after maximum misses
if self.misses >= MAX_MISSES:
    self.game_over = True

def draw(self, surface, font):
    from game import renderer

    # Draw basket and falling objects
    renderer.draw_scene(
        surface,
        self.basket,
        self.objects
    )

    # Draw score
    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    # Draw misses
    renderer.draw_text(
        surface,
        font,
        f"Misses: {self.misses}/{MAX_MISSES}",
        (10, 36)
    )
```

```
# Draw game-over message
if self.game_over:
    renderer.draw_banner(
        surface,
        font,
        f"Game Over! Final score: {self.score}. Press R to re:
    )
```

The important boundary fix

Your old code was:

 Python

```
self.basket.x = max(0, min(WIDTH, self.basket.x))
```

That treats `x` as the **left edge**, but your `Basket` class treats `x` as the **center**.

The corrected version:

 Python

```
half_width = self.basket.width / 2

self.basket.x = max(
    half_width,
    min(WIDTH - half_width, self.basket.x)
)
```

keeps the **whole 90-pixel basket** inside the screen.

And movement is already smooth because `keys_pressed[pygame.K_LEFT] / RIGHT` are checked every frame while the key is held.

i meant `basket.py`

Ah, yes — you meant the **full corrected `basket.py`** . Since the boundary logic belongs in `game.py` 's `handle_input` , the `Basket` class itself mainly needs to remain responsible for its dimensions, position, and rectangle.

Here is the complete `basket.py` :

Corrected `basket.py`



```
"""
```

```
Basket: the player-controlled catcher at the bottom of the screen.
```

```
"""
```

```
import pygame
```

```
class Basket:
```

```
    def __init__(self, x, y, width=90, height=24, speed=5):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        # Normal movement speed
```

```
        self.speed = speed
```

```
        # Speed boost support
```

```
        self.boosted_frames = 0
```

```
        self.normal_speed = speed
```

```
        self.boost_speed = speed
```

```
    def get_rect(self):
```

```
        """
```

```
        Return the pygame rectangle representing the basket.
```

```
        self.x and self.y represent the CENTER of the basket.
```

```
        """
```

```
        return pygame.Rect(
```

```
            int(self.x - self.width / 2),
```

```
            int(self.y - self.height / 2),
```

```
            self.width,
```

```
            self.height,
```

```
        )
```

Important

The **boundary fix** does not need to be inside **basket.py** with your current architecture.

Keep this in **game.py** :

</> Python

```
half_width = self.basket.width / 2

self.basket.x = max(
    half_width,
    min(WIDTH - half_width, self.basket.x)
)
```

So the responsibilities are:

</> Plain text

```
basket.py
    ↓
Stores basket position, size, speed
Provides get_rect()

game.py
    ↓
Reads keyboard
Moves basket
Keeps basket inside screen
```

This is the cleanest structure for your current project.

Here is the spawn code from my Pygame catch game. Currently it spawns at a fixed interval with a plain random x. Improve it so that: (1) the interval between spawns varies randomly, (2) x positions vary but never fall outside the screen (account for the object's radius), (3) a new object doesn't spawn too close to the previous one's x, and (4) there is a maximum number of objects on screen at once. Use named constants so I can tune the values. Explain your changes.

"""

GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic

level (Tasks 2 and 3 ask you to improve them), there's no speed boost

yet (Task 4 builds it from scratch), and catch detection has two known bugs (see game/collision.py and the catch-checking loop

below)

that Task 1 asks you to fix.

"""

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
SPAWN_INTERVAL_FRAMES = 50
```

```
MAX_MISSES = 5
```

```
class GameEngine:
```

```
    def __init__(self):
```

```
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
```

```
        self.objects = []
```

```
        self.frames_until_spawn = 0
```

```
        self.score = 0
```

```
        self.misses = 0
```

```
        self.game_over = False
```

```
    def _spawn_object(self):
```

```
        x = random.randint(20, WIDTH - 20)
```

```
        self.objects.append(FallingObject(x=x, y=-14, speed=3))
```

```
    def handle_input(self, keys_pressed):
```

```
        if self.game_over:
```

```
            return
```

```
        if keys_pressed[pygame.K_LEFT]:
```

```
            self.basket.x -= self.basket.speed
```

```
        if keys_pressed[pygame.K_RIGHT]:
```

```
            self.basket.x += self.basket.speed
```

```
        # Keep the basket inside the screen.
```

```
self.basket.x = max(0, min(WIDTH, self.basket.x))

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
        return

    # Spawn objects
    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

    # Move all falling objects
    for obj in self.objects:
        obj.update()

    # Check which objects were caught
    basket_rect = self.basket.get_rect()

    # IMPORTANT:
    # Iterate over a copy so removing an object does not
    # cause the next object to be skipped.
    for obj in self.objects[:]:
        if is_caught(basket_rect, obj):
            self.score += 1
            self.objects.remove(obj)

    # Check for missed objects
    missed = [
        obj
        for obj in self.objects
        if obj.is_past_bottom(HEIGHT)
    ]

    if missed:
```

```
self.objects = [  
    obj  
    for obj in self.objects  
    if not obj.is_past_bottom(HEIGHT)  
]  
  
self.misses += len(missed)  
  
if self.misses >= MAX_MISSES:  
    self.game_over = True  
  
def draw(self, surface, font):  
    from game import renderer  
  
    renderer.draw_scene(  
        surface,  
        self.basket,  
        self.objects  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        f"Score: {self.score}",  
        (10, 10)  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        f"Misses: {self.misses}/{MAX_MISSES}",  
        (10, 36)  
    )  
  
    if self.game_over:  
        renderer.draw_banner(  
            surface,  
            font,  
            f"Game Over! Final score: {self.score}. Press R to
```

```
restart."
    )
```

Yes. We can make the spawning much more configurable and avoid objects clustering together.

Below is the **full corrected game.py** . I'm assuming your `FallingObject` has a `radius` attribute; if its radius is named differently, change `OBJECT_RADIUS` .

Improved game.py spawning



```
"""
```

```
GameEngine: owns the basket and all falling objects.
```

```
Improved spawning:
```

- Random spawn interval
- Random X position within screen boundaries
- Minimum horizontal distance between consecutive objects
- Maximum number of objects on screen

```
"""
```

```
import random
```

```
import pygame
```

```
from game.basket import Basket
```

```
from game.falling_object import FallingObject
```

```
from game.collision import is_caught
```

```
from game.renderer import WIDTH, HEIGHT
```

```
# =====
```

```
# GAME SETTINGS
```

```
# =====
```

```
# Minimum and maximum number of frames between spawns.
```

```
MIN_SPAWN_INTERVAL_FRAMES = 30
```

```
MAX_SPAWN_INTERVAL_FRAMES = 70
```

```
# Maximum number of falling objects allowed on screen.
```

```
MAX_OBJECTS = 5
```

```
# Radius of a falling object.
# Change this if your FallingObject uses a different radius.
OBJECT_RADIUS = 14

# Minimum horizontal distance between consecutive objects.
MIN_SPAWN_DISTANCE = 60

# Falling object speed.
OBJECT_SPEED = 3

# Maximum number of misses before game over.
MAX_MISSES = 5

class GameEngine:
    def __init__(self):
        self.basket = Basket(
            x=WIDTH / 2,
            y=HEIGHT - 30
        )

        self.objects = []

        # Start with a random spawn delay.
        self.frames_until_spawn = random.randint(
            MIN_SPAWN_INTERVAL_FRAMES,
            MAX_SPAWN_INTERVAL_FRAMES
        )

        self.score = 0
        self.misses = 0
        self.game_over = False

    def _get_spawn_x(self):
        """
        Generate a valid X position for a new object.

        The position:
        1. Stays inside the screen.
        2. Is at least MIN_SPAWN_DISTANCE away from
```

```
        the previous object's X position.
        """

    # Object center must stay at least OBJECT_RADIUS
    # away from either edge of the screen.
    min_x = OBJECT_RADIUS
    max_x = WIDTH - OBJECT_RADIUS

    # If there are no existing objects, any valid X is okay.
    if not self.objects:
        return random.randint(min_x, max_x)

    # Get the X position of the most recently spawned object.
    previous_x = self.objects[-1].x

    # Try several random positions until we find one
    # sufficiently far away.
    for _ in range(20):
        x = random.randint(min_x, max_x)

        if abs(x - previous_x) >= MIN_SPAWN_DISTANCE:
            return x

    # Fallback:
    # If we couldn't find a suitable random position,
    # choose the side of the screen farther from the
    # previous object.
    left_distance = abs(previous_x - min_x)
    right_distance = abs(max_x - previous_x)

    if left_distance >= right_distance:
        return min_x
    else:
        return max_x

def _spawn_object(self):
    """
    Spawn a new falling object if the screen isn't full.
    """
```

```
# Don't spawn if the maximum number of objects
# is already on screen.
if len(self.objects) >= MAX_OBJECTS:
    return

x = self._get_spawn_x()

self.objects.append(
    FallingObject(
        x=x,
        y=-OBJECT_RADIUS,
        speed=OBJECT_SPEED
    )
)

def handle_input(self, keys_pressed):
    if self.game_over:
        return

    # Move basket continuously while the key is held.
    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed

    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed

    # Keep the entire basket inside the screen.
    half_width = self.basket.width / 2

    self.basket.x = max(
        half_width,
        min(WIDTH - half_width, self.basket.x)
    )

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()

def update(self):
    if self.game_over:
```

```
        return

# =====
# SPAWNING
# =====

self.frames_until_spawn -= 1

if self.frames_until_spawn <= 0:

    # Only spawn if we haven't reached the limit.
    if len(self.objects) < MAX_OBJECTS:
        self._spawn_object()

    # Pick a NEW random interval for the next spawn.
    self.frames_until_spawn = random.randint(
        MIN_SPAWN_INTERVAL_FRAMES,
        MAX_SPAWN_INTERVAL_FRAMES
    )

# =====
# UPDATE OBJECTS
# =====

for obj in self.objects:
    obj.update()

# =====
# CATCH DETECTION
# =====

basket_rect = self.basket.get_rect()

# Iterate over a copy so removing an object doesn't
# cause the next object to be skipped.
for obj in self.objects[:]:
    if is_caught(basket_rect, obj):
        self.score += 1
        self.objects.remove(obj)
```

```
# =====
# MISSED OBJECTS
# =====

missed = [
    obj
    for obj in self.objects
    if obj.is_past_bottom(HEIGHT)
]

if missed:
    self.objects = [
        obj
        for obj in self.objects
        if not obj.is_past_bottom(HEIGHT)
    ]

    self.misses += len(missed)

    if self.misses >= MAX_MISSES:
        self.game_over = True

def draw(self, surface, font):
    from game import renderer

    renderer.draw_scene(
        surface,
        self.basket,
        self.objects
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
```

```

        font,
        f"Misses: {self.misses}/{MAX_MISSES}",
        (10, 36)
    )

    if self.game_over:
        renderer.draw_banner(
            surface,
            font,
            f"Game Over! Final score: {self.score}. Press R to re:
        )

```

What changed?

1. Random spawn interval

Instead of:

</> Python

```
SPAWN_INTERVAL_FRAMES = 50
```

we now have:

</> Python

```
MIN_SPAWN_INTERVAL_FRAMES = 30
MAX_SPAWN_INTERVAL_FRAMES = 70
```

Every time an object spawns:

</> Python

```
self.frames_until_spawn = random.randint(
    MIN_SPAWN_INTERVAL_FRAMES,
    MAX_SPAWN_INTERVAL_FRAMES
)
```

So you might get:

</> Plain text

30 frames

↓

57 frames

↓

42 frames

↓

68 frames

↓

35 frames

This makes the game less predictable.

2. Object stays inside the screen

Previously:

 Python

```
x = random.randint(20, WIDTH - 20)
```

Now:

 Python

```
min_x = OBJECT_RADIUS  
max_x = WIDTH - OBJECT_RADIUS
```

If:

 Plain text

```
WIDTH = 800  
OBJECT_RADIUS = 14
```

then the object's center can only be between:

 Plain text

14 ————— 786

This prevents the object from spawning partially outside the screen.

3. Objects don't spawn too close together

We remember the previous object's X:

</> Python

```
previous_x = self.objects[-1].x
```

Then require:

</> Python

```
abs(x - previous_x) >= MIN_SPAWN_DISTANCE
```

With:

</> Python

```
MIN_SPAWN_DISTANCE = 60
```

if the previous object was at:

</> Plain text

```
x = 300
```

the next one cannot spawn between:

</> Plain text

```
240 ————— 360
```

So you'll get something more like:

</> Plain text

```
Object 1: x = 300  
Object 2: x = 125  
Object 3: x = 500  
Object 4: x = 250
```

rather than:

`</>` Plain text

```
300
305
297
310
```

4. Maximum objects on screen

We added:

`</>` Python

```
MAX_OBJECTS = 5
```

and before spawning:

`</>` Python

```
if len(self.objects) >= MAX_OBJECTS:
    return
```

So the game can never have more than **5 falling objects simultaneously**.

Values you can easily tune

At the top of the file:

`</>` Python

```
MIN_SPAWN_INTERVAL_FRAMES = 30
MAX_SPAWN_INTERVAL_FRAMES = 70

MAX_OBJECTS = 5

OBJECT_RADIUS = 14

MIN_SPAWN_DISTANCE = 60

OBJECT_SPEED = 3
```

For example, to make the game **harder**:

`</>` Python

```
MIN_SPAWN_INTERVAL_FRAMES = 20
MAX_SPAWN_INTERVAL_FRAMES = 45
MAX_OBJECTS = 8
OBJECT_SPEED = 4
```

To make it **easier**:

`</>` Python

```
MIN_SPAWN_INTERVAL_FRAMES = 50
MAX_SPAWN_INTERVAL_FRAMES = 90
MAX_OBJECTS = 3
OBJECT_SPEED = 2
```

One important point: if your `FallingObject` class already defines something like `radius = 14`, we should use that instead of duplicating `OBJECT_RADIUS`. If you send me `falling_object.py`, I can make the spawning code match it exactly.

Here are my `Basket` class, the engine's `handle_keydown` and `draw` methods, and the renderer's `draw_scene`. Add a temporary speed boost: the player presses Space to activate it, the basket moves faster for about 3 seconds, there is a clear visual indication while it's active (HUD text with a countdown and a basket color change), and it automatically returns to normal speed when it expires. Use the existing `boosted_frames`, `normal_speed` and `boost_speed` fields in `Basket`. Optionally add a cooldown so it can't be spammed. Explain your changes and tell me which files to edit.

Yes. You already have the right fields in `Basket`, so we can build the boost without changing the overall architecture.

You need to edit **3 files**:

1. `game/basket.py` — add boost activation/update logic.
2. `game/game_engine.py` — detect Space and update the boost every frame.
3. `game/renderer.py` — change the basket color and display the countdown.

I recommend a **3-second boost at 60 FPS = 180 frames**, with a **5-second cooldown** so Space can't be spammed.

1. `game/basket.py`

Replace your current class with:

Updated basket.py



"""

Basket: the player-controlled catcher at the bottom of the screen.

"""

```
import pygame
```

```
# Boost configuration
```

```
BOOST_DURATION_FRAMES = 180    # 3 seconds at 60 FPS
```

```
BOOST_COOLDOWN_FRAMES = 300    # 5 seconds at 60 FPS
```

```
class Basket:
```

```
    def __init__(self, x, y, width=90, height=24, speed=5):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.width = width
```

```
        self.height = height
```

```
        # Normal movement speed
```

```
        self.speed = speed
```

```
        # Speed boost settings
```

```
        self.boosted_frames = 0
```

```
        self.normal_speed = speed
```

```
        self.boost_speed = speed * 2
```

```
        # Cooldown prevents the boost from being spammed
```

```
        self.boost_cooldown_frames = 0
```

```
    def get_rect(self):
```

```
        """
```

```
        Return the pygame rectangle representing the basket.
```

```
        self.x and self.y represent the CENTER of the basket.
```

```
        """
```

```
        return pygame.Rect(
```

```
        int(self.x - self.width / 2),
        int(self.y - self.height / 2),
        self.width,
        self.height,
    )

def activate_boost(self):
    """
    Activate the speed boost if it isn't already active
    and the cooldown has expired.
    """
    if self.boosted_frames > 0:
        return False

    if self.boost_cooldown_frames > 0:
        return False

    self.boosted_frames = BOOST_DURATION_FRAMES
    self.speed = self.boost_speed

    return True

def update_boost(self):
    """
    Update the boost timer and automatically restore
    normal speed when the boost expires.
    """

    # Update boost
    if self.boosted_frames > 0:
        self.boosted_frames -= 1

        if self.boosted_frames <= 0:
            self.boosted_frames = 0
            self.speed = self.normal_speed

            # Start cooldown when boost expires
            self.boost_cooldown_frames = BOOST_COOLDOWN_FRAMES

    # Update cooldown
```

```

        if self.boost_cooldown_frames > 0:
            self.boost_cooldown_frames -= 1

    def is_boost_active(self):
        """Return True while the speed boost is active."""
        return self.boosted_frames > 0

    def get_boost_seconds(self):
        """Return remaining boost time in seconds."""
        return self.boosted_frames / 60

```

2. game/game_engine.py

You need to modify **three** places.

Update `handle_keydown`

Replace your current method with:

`</>` Python

```

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self.__init__()
        return

    if not self.game_over and key == pygame.K_SPACE:
        self.basket.activate_boost()

```

Now:

`</>` Plain text

```

SPACE pressed
↓
activate_boost()
↓
Is boost already active?
↓
Is cooldown active?
↓
No → activate

```

↓

```
speed = boost_speed
```

Update update()

At the beginning of `update()` , after the game-over check:

Python

```
def update(self):
    if self.game_over:
        return

    # Update basket speed boost
    self.basket.update_boost()

    # ...rest of your existing update code...
```

So your beginning should look like:

Python

```
def update(self):
    if self.game_over:
        return

    # Update temporary speed boost
    self.basket.update_boost()

    # Spawn objects
    self.frames_until_spawn -= 1

    if self.frames_until_spawn <= 0:
        self._spawn_object()
        self.frames_until_spawn = random.randint(
            MIN_SPAWN_INTERVAL_FRAMES,
            MAX_SPAWN_INTERVAL_FRAMES
        )

    # Update falling objects
    for obj in self.objects:
        obj.update()
```

The important part is that `update_boost()` runs **every frame**.

3. game/renderer.py

Your `draw_scene()` currently draws the basket.

Change it so that it checks whether the boost is active.

For example:

`</>` Python

```
def draw_scene(surface, basket, objects):
    surface.fill(BACKGROUND_COLOR)

    # Draw falling objects
    for obj in objects:
        obj.draw(surface)

    # Change basket color while boost is active
    if basket.is_boost_active():
        basket_color = (255, 215, 0)    # Gold
    else:
        basket_color = (100, 150, 255) # Normal blue

    basket_rect = basket.get_rect()

    pygame.draw.rect(
        surface,
        basket_color,
        basket_rect
    )
```

If your existing `draw_scene()` already has other rendering code, **don't replace the whole function**. Just change the basket-drawing portion.

Add the boost HUD

Wherever your renderer has `draw_text()`, add:

`</>` Python

```
if basket.is_boost_active():
    remaining = max(0, basket.get_boost_seconds())

    draw_text(
        surface,
        font,
        f"B00ST: {remaining:.1f}s",
```

```
(10, 62)  
)
```

If your `draw_scene()` doesn't receive `font` , it's cleaner to put this in your `GameEngine.draw()` instead.

Your `GameEngine.draw()` can become:

</> Python

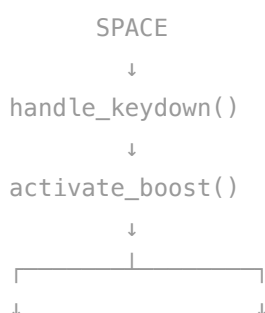
```
def draw(self, surface, font):  
    from game import renderer  
  
    renderer.draw_scene(  
        surface,  
        self.basket,  
        self.objects  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        f"Score: {self.score}",  
        (10, 10)  
    )  
  
    renderer.draw_text(  
        surface,  
        font,  
        f"Misses: {self.misses}/{MAX_MISSES}",
```

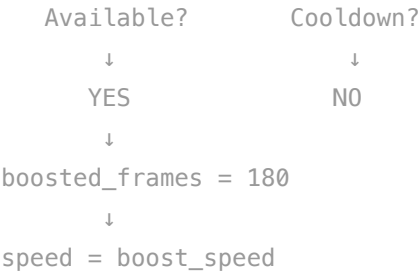
This is actually preferable because your renderer doesn't need to know about game state.

How it works

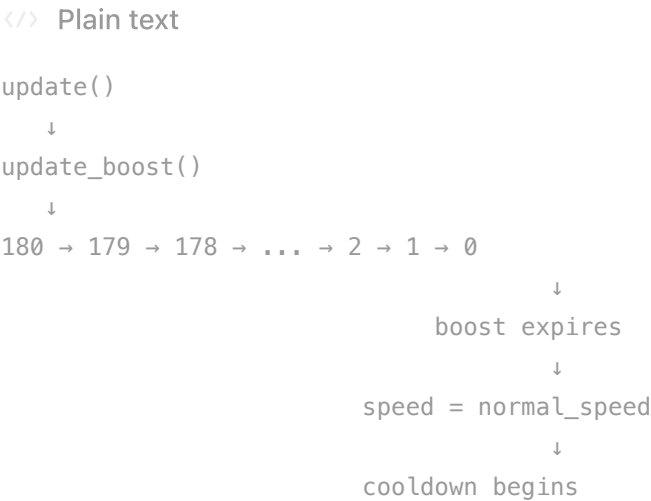
When the player presses **Space**:

</> Plain text



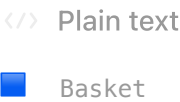


Then every game frame:

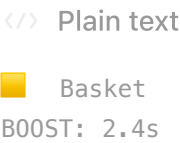


Visual feedback

Normal:



Boost active:



So the player immediately knows that the boost is active.

One important detail

You currently have:



```
self.boost_speed = speed
```

in your original `Basket` .

That means the boost would **not actually** make the basket faster.

Change it to:

```
</> Python
```

```
self.boost_speed = speed * 2
```

So with:

```
</> Python
```

```
speed = 5
```

you get:

```
</> Plain text
```

```
Normal speed = 5
Boost speed   = 10
Duration      = 3 seconds
Cooldown     = 5 seconds
```

You can tune everything from the top of `basket.py` :

```
</> Python
```

```
BOOST_DURATION_FRAMES = 180
BOOST_COOLDOWN_FRAMES = 300
```

and:

```
</> Python
```

```
self.boost_speed = speed * 2
```

For example, `speed * 1.5` gives a more subtle boost, while `speed * 3` makes it much stronger.

```
python main.py
pygame-ce 2.5.8 (SDL 2.32.10, Python 3.14.5)
Traceback (most recent call last):
  File "/Users/sanjayvarma/Desktop/se/07_catch_falling/catch-falling/main.py", line 43, in <module>
    main()
    ~~~~^^
  File "/Users/sanjayvarma/Desktop/se/07_catch_falling/catch-falling/main.py", line 33, in main
    engine.update()
    ~~~~~~^
  File "/Users/sanjayvarma/Desktop/se/07_catch_falling/catch-falling/game/game_engine.py", line 188, in update
    if is_caught(basket_rect, obj):
        ~~~~~~^
  File "/Users/sanjayvarma/Desktop/se/07_catch_falling/catch-falling/game/collision.py", line 7, in is_caught
    obj_rect = obj.get_rect()
                ^
AttributeError: 'FallingObject' object has no attribute 'get_rect'
(venv) sanjayvarma@Sanjays-MacBook-Pro-10 catch-falling %
```

The error is straightforward: my earlier `collision.py` assumed that `FallingObject` had a `get_rect()` method, but your `FallingObject` class doesn't.

So don't add `get_rect()` just for this unless you want to. The cleaner fix is to make `collision.py` use the object's existing `x`, `y`, and `radius`.

Replace `game/collision.py` with this

Corrected collision.py



```
"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    """
    Return True if the falling object overlaps the basket.
```

The object is treated as a circle using its x, y and radius.

"""

```
# Falling object's horizontal bounds
```

```
obj_left = obj.x - obj.radius
```

```
obj_right = obj.x + obj.radius
```

```
# Falling object's vertical bounds
```

```
obj_top = obj.y - obj.radius
```

```
obj_bottom = obj.y + obj.radius
```

```
# Check horizontal overlap
```

```
horizontal_overlap = (  
    obj_right >= basket_rect.left  
    and obj_left <= basket_rect.right  
)
```

```
# Check vertical overlap
```

```
vertical_overlap = (  
    obj_bottom >= basket_rect.top  
    and obj_top <= basket_rect.bottom  
)
```

```
return horizontal_overlap and vertical_overlap
```

Why this fixes both problems

Your old code was:

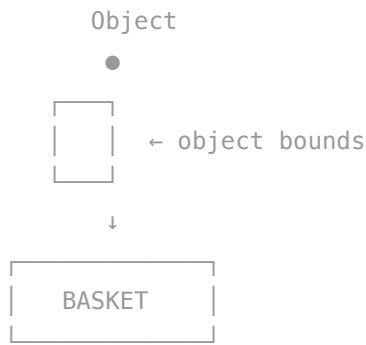
</> Python

```
return basket_rect.left <= obj.x <= basket_rect.right
```

That only checked X, so an object could be caught while it was still at the top of the screen.

The new code checks both:

</> Plain text



The object must overlap the basket **horizontally AND vertically**.

Your other fix is still correct

In `game_engine.py`, keep:

`</>` Python

```
for obj in self.objects[:]:
    if is_caught(basket_rect, obj):
        self.score += 1
        self.objects.remove(obj)
```

The `[:]` is important because it prevents the "one object gets skipped" bug when removing objects during iteration.

One thing to verify

This fix assumes your `FallingObject` has:

`</>` Python

```
self.x
self.y
self.radius
```

Given your spawning code:

`</>` Python

```
FallingObject(x=x, y=-14, speed=3)
```

it very likely does have a radius, but if you run it and get:

 Plain text

```
AttributeError: 'FallingObject' object has no attribute 'radius'
```

then send me your **game/falling_object.py** and I'll give you the exact collision code matching your class.